

LEC 2

Silent Failures & Automated Validation

Bad inputs in AI cause silent failures, not crashes

```
from pydantic import BaseModel, Field, ValidationError

class UserDataInput(BaseModel):
    age: int = Field(..., ge=0, le=120) # Age must be 0-120
    income: float = Field(..., ge=0.0) # Income cannot be negative
    user_segment: str

# If bad data arrives in the pipeline, it raises an error BEFORE hitting the AI:
try:
    bad_data = UserDataInput(age=150, income=-500, user_segment="A")
except ValidationError as e:
    print("Pipeline halted: Data contract violated!")
```

The AI Reproducibility Crisis & DVC

Model State = Code + Data + Environment

Git tracks code perfectly, but fails on 50GB datasets

```
# 1. Track massive dataset with DVC (hashes the data)
dvc add data/training_data.parquet

# 2. Track the lightweight DVC pointer with Git
git add data/training_data.parquet.dvc
git commit -m "Added Week 2 training dataset"

# 3. Push heavy data to cloud storage (S3/GCS)
dvc push
```

Lec 5

Logic Programming & Symbolic Rules — Guardrails

Symbolic AI

Hard-coded knowledge, deterministic rules, always predictable.

Neuro-Symbolic

Combines pattern recognition (ML) with reasoning (rules).

Pipeline Guardrails

Rules act as safety filters before and after ML predictions.

```
def safe_loan_prediction(user_data, ml_model):  
    # 1. Symbolic Guardrail — Hard Constraint  
    if user_data['credit_score'] < 500:  
        return {"status": "Rejected",  
                "reason": "Policy: Min credit score"}  
  
    # 2. ML Model Prediction  
    prediction = ml_model.predict(user_data)  
  
    # 3. Post-Prediction Sanity Check  
    if prediction == 'Approved' \  
        and user_data['amount'] > 1_000_000:  
        return {"status": "Manual Review"}  
  
    return {"status": prediction}
```

Raw Data



Rule Engine



ML Model



Output

Introduction to LIME

LIME

Local Interpretable Model-agnostic Explanations

Focus

Strictly local explainability

Approach

Model-agnostic — works with any `.predict()` method

Mechanism

Perturbs input, observes prediction changes, fits a local linear model

```
import lime.lime_tabular  
  
# 1. Initialize the Explainer  
explainer = lime.lime_tabular.LimeTabularExplainer(  
    training_data = X_train.values,  
    feature_names = X_train.columns,  
    mode = 'classification'  
)  
  
# 2. Explain a single prediction  
exp = explainer.explain_instance(  
    data_row = X_test.iloc[0].values,  
    predict_fn = rf_model.predict_proba  
)  
  
# 3. Show explanation  
exp.show_in_notebook()
```

Lec6

Confusion Matrix — Code Example

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

y_true = [1, 0, 1, 1, 0, 1]
y_pred = [1, 0, 0, 1, 0, 1]

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.show()
```

Why Use It?

Better than a single score — reveals the shape of errors, not just their count.

TP + TN

Correct predictions. These are what we want to maximise.

FP + FN

Prediction errors. Identify which type matters more for your task.

sklearn.metrics.confusion_matrix() is the quickest path to actionable error insight.

Configuration-Driven Experiments

```
import yaml
import mlflow

with open("config.yaml") as f:
    cfg = yaml.safe_load(f)

mlflow.set_experiment(cfg["experiment_name"])

with mlflow.start_run():
    mlflow.log_params(cfg["model"])
    mlflow.log_param("train_size",
                    cfg["data"]["train_size"])
    mlflow.log_metric("f1", 0.87)
```

config.yaml

```
experiment_name: run_01
model:
  lr: 0.001
  epochs: 30
data:
  train_size: 0.8
```

Benefits

- Rerun any experiment with one command
- Audit differences between runs easily
- No accidental hardcoded changes
- MLflow tracks params, metrics & artifacts

Factory Pattern in AI Systems

```
config.yaml  
experiment: run_03
```

Factory

Reads config → creates objects

```
create_model("random_forest")
```

```
create_metric("f1")
```

```
create_dataset("csv")
```

```
model = XGBoost()
```

Centralised Creation

All object creation in one place — no hardcoded class names scattered across the codebase.

```
class ModelFactory:  
    @staticmethod  
    def create(model_type):  
        if model_type == "xgb":  
            return XGBoostModel()  
        elif model_type == "rf":  
            return RandomForestModel()
```

Extensible

Add a new model type by registering it — no changes to callers or pipeline code needed.

Config-Driven

The config file drives which classes are instantiated. Swap models without touching code.

Strategy Pattern — Code Example

```
if loss == "mse":  
    ...  
elif loss == "mae":  
    ...  
  
class LossStrategy:  
    def compute(self, y_true, y_pred):  
        pass  
  
class MSE(LossStrategy):  
    def compute(self, y_true, y_pred):  
        return ((y_true - y_pred) ** 2).mean()  
  
class MAE(LossStrategy):  
    def compute(self, y_true, y_pred):  
        return abs(y_true - y_pred).mean()
```

EvalStrategy
(Abstract)

MSE Strategy

MAE Strategy

Evaluator
(uses strategy)

Implementation: Sliding Windows in Python

Slide 7 · Practical Code

```
feature_engineering.py

import pandas as pd

# 1. Rolling Statistics (Statistical Feature)
df['temp_ma'] = df['temp'].rolling(window=5).mean()

# 2. Lagging (Sequential Feature for RNNs)
df['prev_val'] = df['temp'].shift(1)

# 3. Differencing (Stationarity for ANOVA)
df['delta'] = df['temp'].diff()
```

Rolling / Smoothing

Computes a moving average over a window of 5 samples — reduces noise.

Lagging / History

Creates a shifted version of the column — provides the model its prior value.

Differencing / Change

Computes the delta between consecutive rows — key for stationarity in ANOVA.